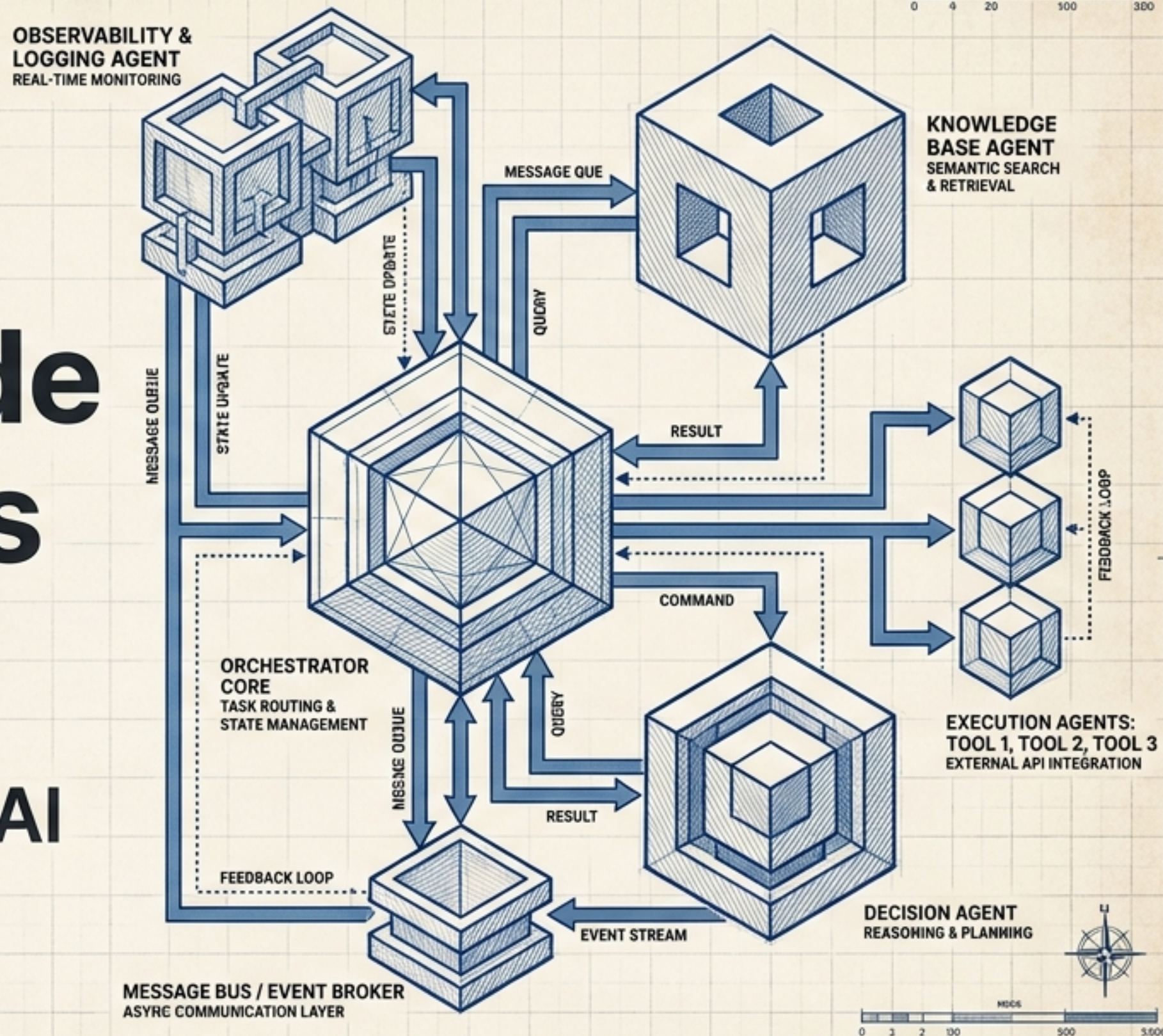


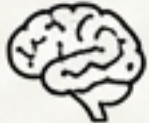
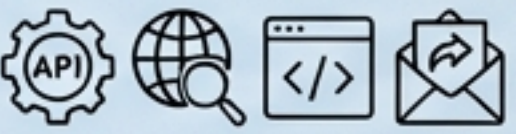
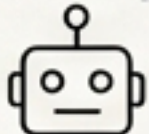
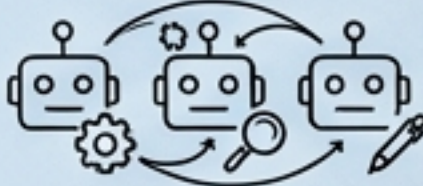
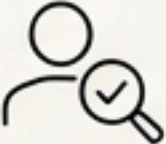
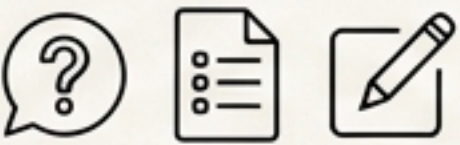
Building Production-Grade Agentic Systems

A Builder's Blueprint for Asynchronous Multi-Agent Frameworks using the OpenAI Agents SDK



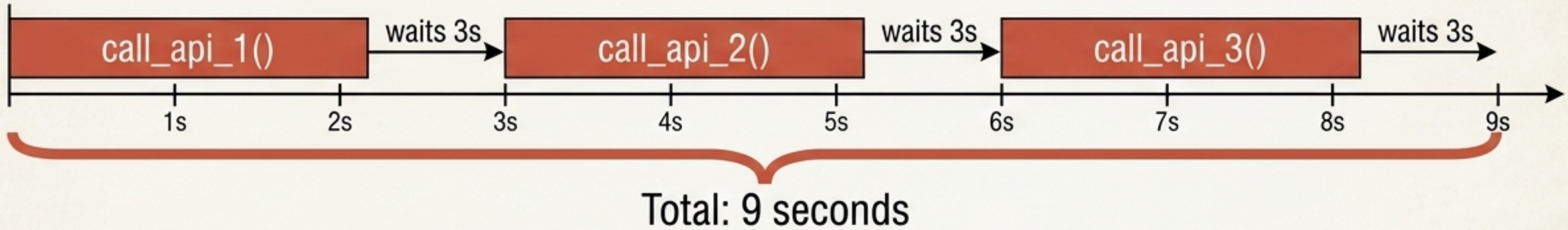
	DRAWING TITLE: ASYNCHRONOUS MULTI-AGENT FRAMEWORK	
	PROJECT: OPENAI AGENTS SDK IMPLEMENTATION	
	DRAWING: SYSTEM ARCHITECT	DATE: OCT 26, 2024
		REV: A.1

The Evolution from Stateless Bots to Autonomous Workers

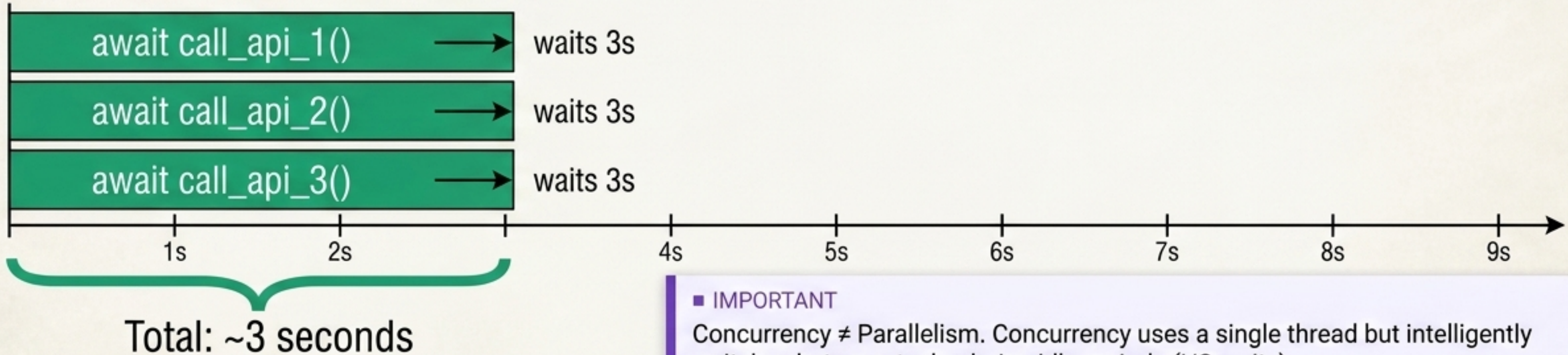
	Traditional LLM	Agentic AI
Interaction Model	Single turn: prompt → response →	Multi-turn: plan → act → observe → adapt ↻
Decision Making	Stateless; no memory between calls 	Stateful; tracks goals, context, history 
Tool Usage	None by default 	Can call APIs, search web, run code, send emails 
Collaboration	Single model 	Multiple specialised agents working together 
Human Oversight	Required for every step 	Minimal; guardrails handle safety automatically 
Core Use Case	Q&A, summarisation, generation 	Complex workflows, research, and sales automation 

Solving the Multi-Agent Latency Bottleneck

Synchronous (Sequential)



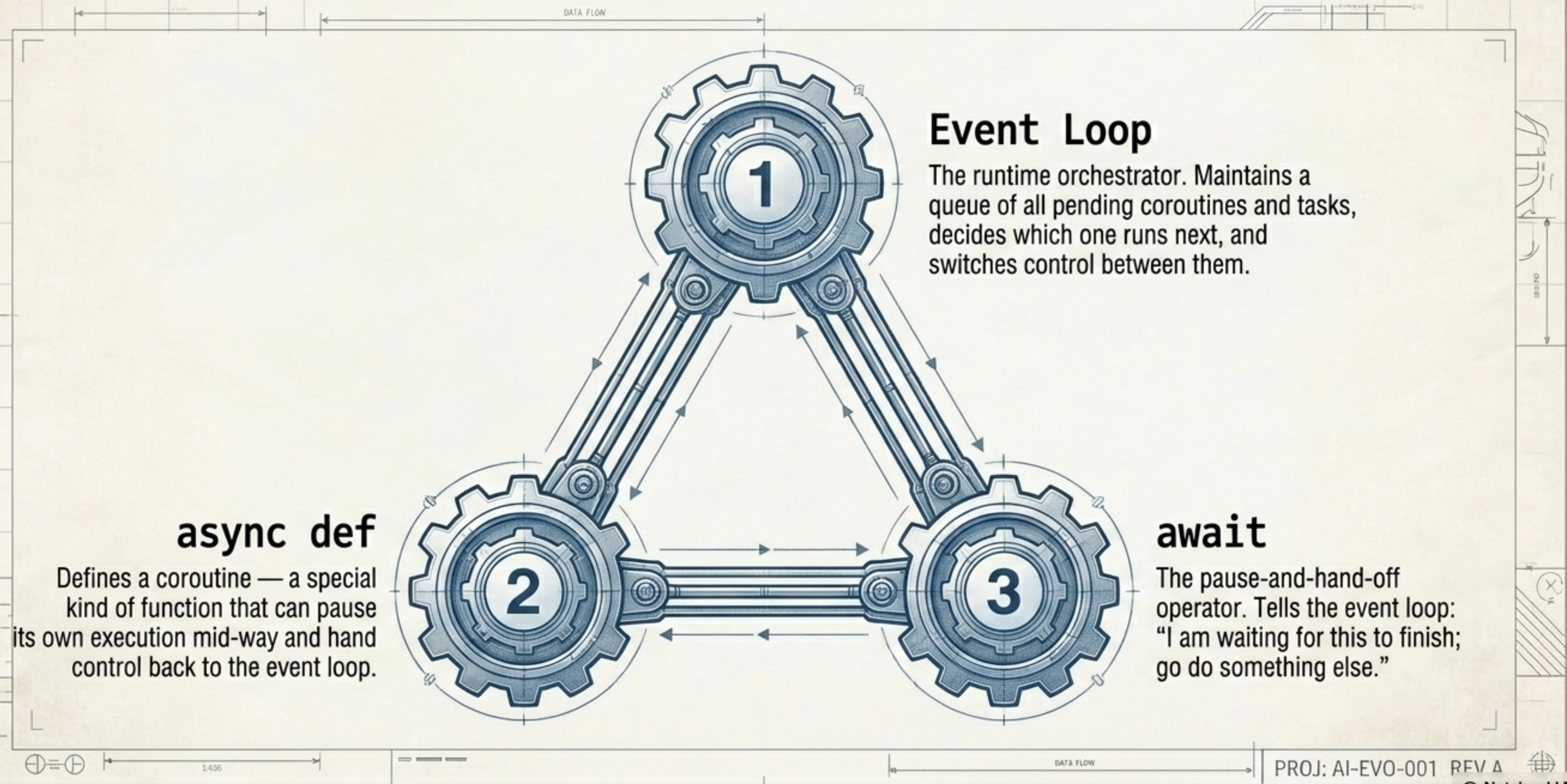
Asynchronous (Concurrent)



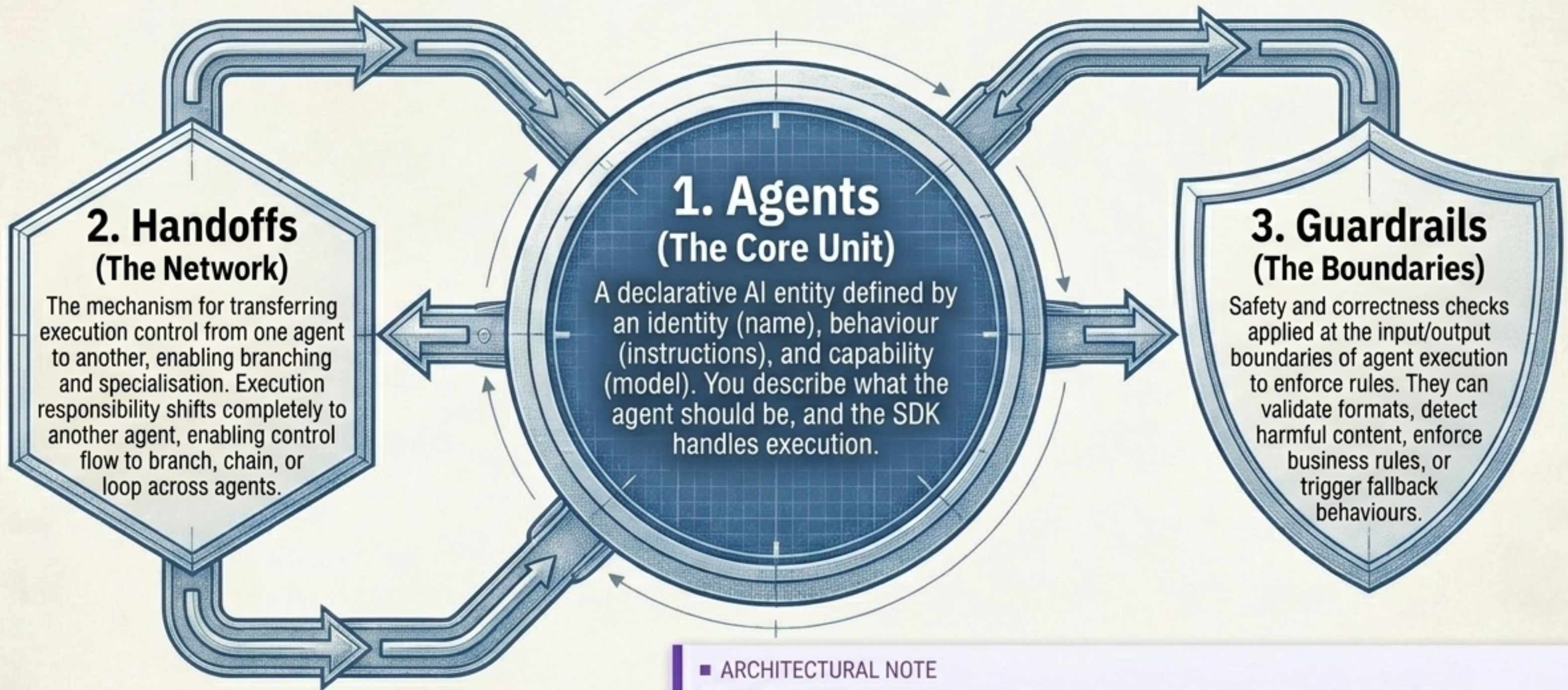
■ IMPORTANT

Concurrency $\neq$ Parallelism. Concurrency uses a single thread but intelligently switches between tasks during idle periods (I/O waits).

The Physics of Concurrency: The AsyncIO Trinity



The OpenAI Agents SDK: Core Architecture Primitives



■ ARCHITECTURAL NOTE

The SDK handles complex orchestration plumbing but remains model-agnostic—it supports Gemini, Groq, and Anthropic alongside OpenAI.

The Three-Step Agent Execution Lifecycle

Step 1: Definition

Instantiate the Agent. Define its name, system prompt instructions, model, tools, and handoffs declaratively.

```
agent = Agent(name='...',  
instructions='...')
```

Step 2: Orchestration

Pass the Agent and the user input to the Runner.

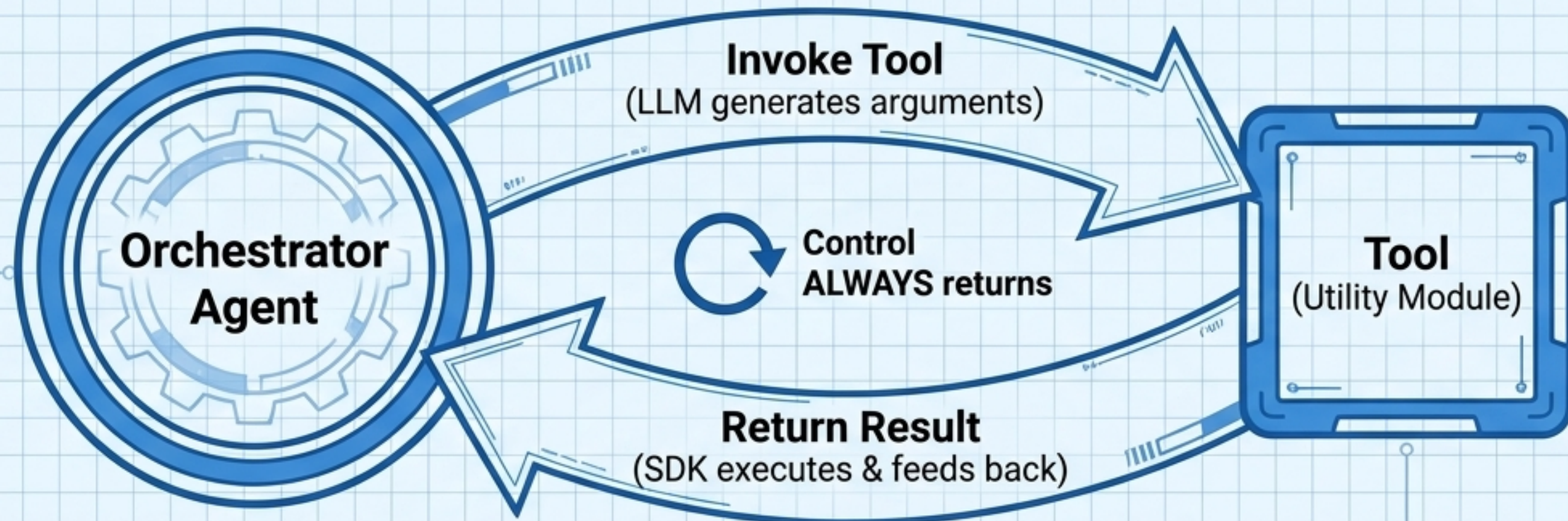
The Runner orchestrates the LLM loop: managing API calls, executing API calls, executing tools, and processing handoffs.

Step 3: Execution

Use `await Runner.run()` to trigger the async execution, ultimately returning a structured `RunResult` object containing the final output.

```
result = await  
Runner.run(agent, input)
```


Extending Capabilities: Tools as the Boomerang



Mechanism

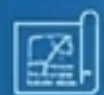
A Python function decorated with `@tool`. The LLM generates JSON arguments, the SDK executes the raw Python, and feeds the result back to the LLM. Contrast with handoffs where responsibility transfers completely.



Ideal For

Interacting with external systems where the main agent must **stay in charge** (e.g., executing web searches, querying databases, triggering SendGrid emails).

Delegating Responsibility: Handoffs as the Relay Baton



Mechanism

The orchestrator registers a specialist agent as a potential target. The LLM decides to transfer responsibility based on the conversation context. Contrast this transfer of responsibility with tools.



Ideal For

Building specialized sequential pipelines, **role-based** delegation, and complex branching workflows (e.g., Triage → Billing Specialist).

The Architecture Diagnostic: Tools vs. Handoffs

Tools

Primary Action:
Invoke and Return

Execution Shift:
None. The Orchestrator stays active.

Real-World Example:



An Agent uses a SendGrid tool to fire off an email, then continues planning its next move.

Handoffs

Primary Action:
Delegate and Exit

Execution Shift:
Complete transfer of responsibility to a specialist.

Real-World Example:



A generic Orchestrator hands off an angry customer to a specialized Billing Agent equipped to process complex refunds.

Eliminating Brittle Parsing: Structured Outputs

*{name: 'Company X',
"pain": "high cost" ...}*



**Pydantic
output_type
Schema**

```
{  
  "company_name": "Company X",  
  "pain_points": [  
    "high cost"  
  ]  
}
```



The Concept:

By default, agents output free-form text. By supplying a Pydantic model to the output_type parameter, the SDK forces the LLM to conform to an exact schema.



The Architecture Value:

This guarantees type-safe data flow. Agent A's structured JSON output feeds flawlessly into Agent B's input. It replaces brittle RegEx parsing and JSON.loads() errors with reliable, native Python objects.

Multi-Model Optimisation: The Right Engine for the Task

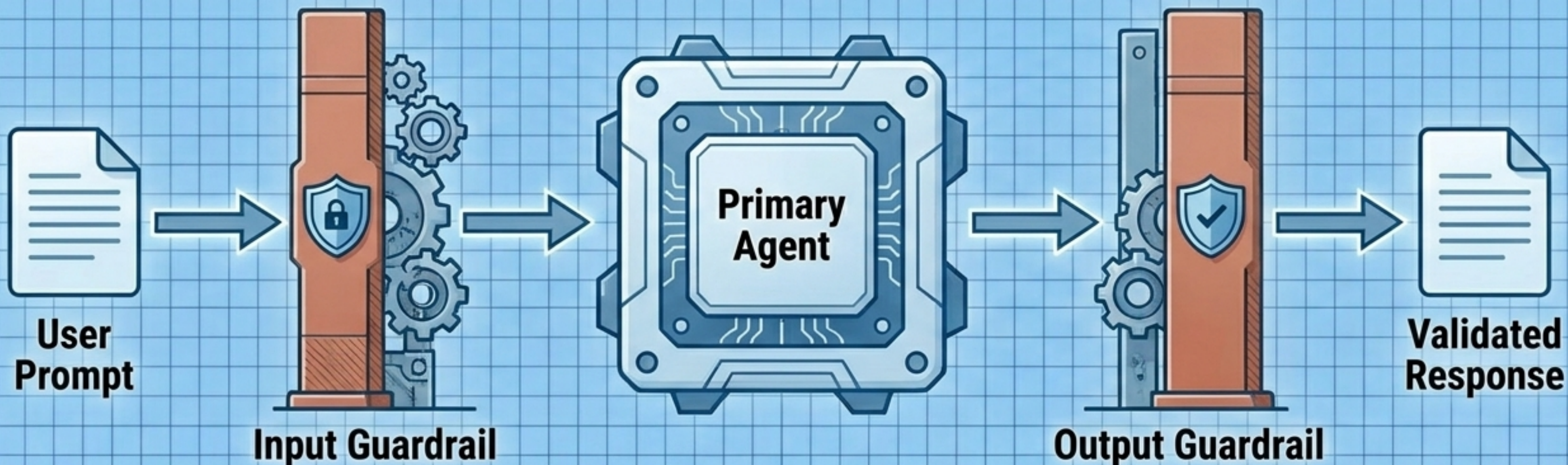
Agent Role	Suggested Model	Architectural Reasoning
Triage / Router Agent	gpt-4o-mini or Gemini Flash	Fast, cheap — only needs to classify intent
Deep Researcher Agent	gpt-4o or Claude Opus	Requires complex reasoning, long context windows, and nuanced analysis
Email Drafter	gpt-4o-mini	Highly cost-effective for creative writing at scale
Validator / Guardrail	Any OpenAI Model	Required due to current SDK constraints



Key Concept:

You do not need to run expensive models for every task. The SDK allows mixing providers and weights in the same pipeline to optimise for cost, speed, and capability simultaneously.

Enterprise Safety: The 'AI Watching AI' Paradigm



Input Guardrails:

Intercept data before execution. Used to block harmful content, reject PII, or stop prompt injections via the `InputGuardrailTripwireTriggered` exception.



Output Guardrails:

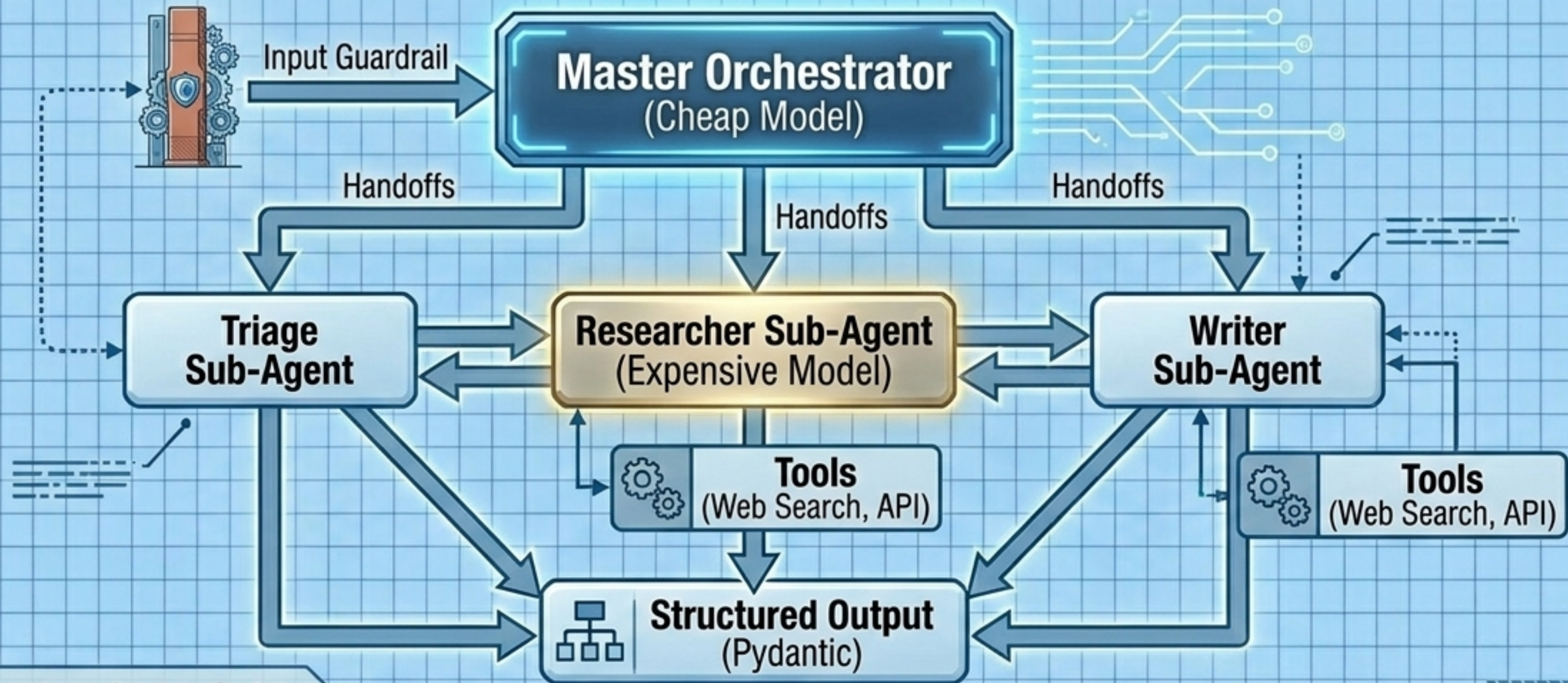
Intercept data after execution. Used to validate tone, schema formats, or enforce business rules.



Architectural Constraint:

Currently, Validator Guardrails rely entirely on OpenAI models, even if the primary working agent is Gemini or Groq.

The Master Blueprint: The Orchestrator Pattern



Synthesis Insight:

Scalable AI is not about clever prompting; it is about modular, type-safe architecture. A single Orchestrator safely routes a request via Handoffs to isolated Specialists, who use precise Tools, and return uniform Structured Outputs.

Project Blueprint: The SDR Agent Pipeline

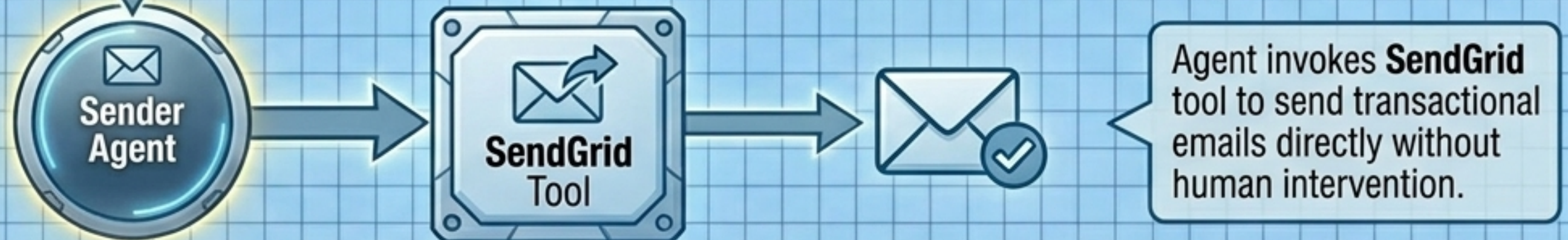
Layer 1: Workflow Design



Layer 2: Agent Calling Other Agents

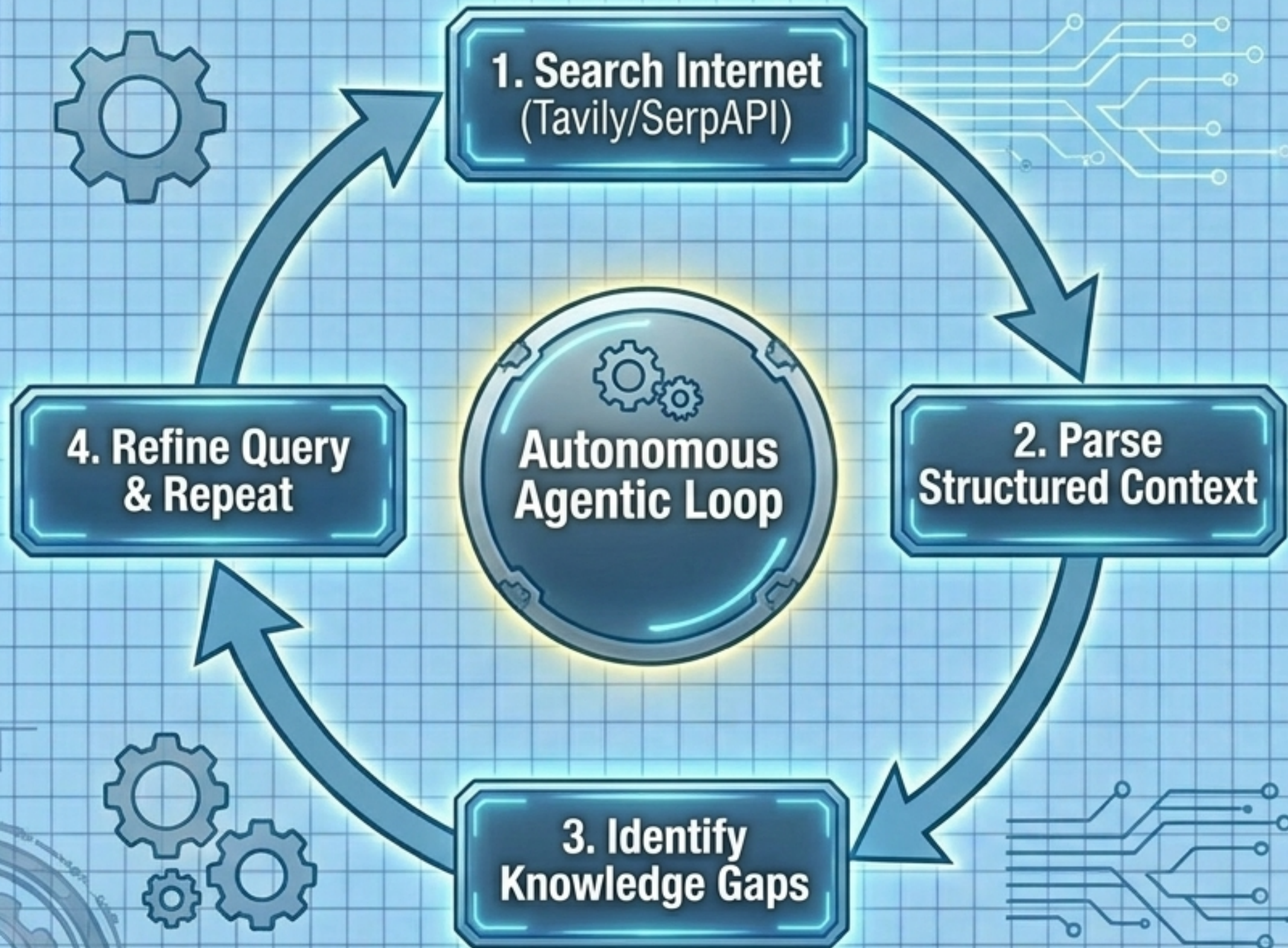


Layer 3: Tool Execution



Result: An autonomous, fully executed B2B sales sequence.

Beyond Linear Pipelines: Autonomous Deep Research



The Capability:

Agentic AI at its most powerful. Not a linear script, but an autonomous entity that directs its own workflow based on intermediate discoveries.

The Loop:

Uses tools to retrieve live info, ingests it, reasons about what is missing, and decides to search again until the goal is fully satisfied.

Final Takeaway:

The OpenAI Agents SDK provides the necessary asynchronous infrastructure, type safety, and delegation mechanics to make these cyclic autonomous systems viable in production.